

TD : 1 Modele géométrique inverse d'un robot Scara

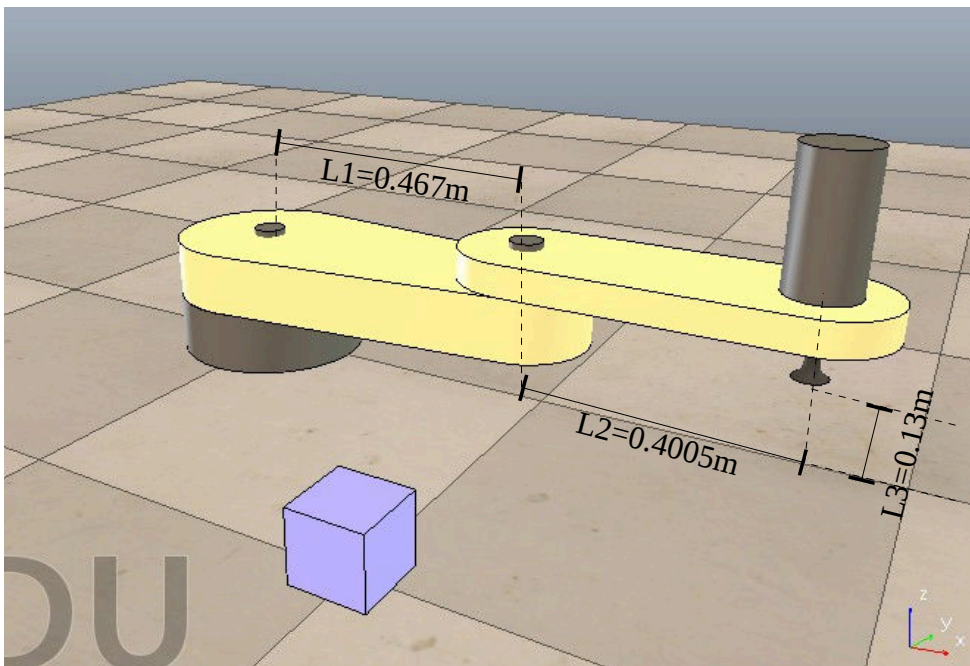
Objectif : Générer une trajectoire pour le robot Scara représenté ci-dessous :

Le robot doit attraper le cube d'arête $0,1m$, pour l'emmener du point $P1$ de coordonnées ${}^0P1 = \begin{bmatrix} 0.5m \\ -0.5m \\ 0.1m \end{bmatrix}$ au

point $P2$ de coordonnées ${}^0P2 = \begin{bmatrix} 0.5m \\ 0.5m \\ 0.1m \end{bmatrix}$, en se déplaçant sur la ligne correspondante, avec une erreur

inférieure à $1mm$. Il doit ensuite revenir au point $P3$ de coordonnées ${}^0P3 = \begin{bmatrix} 0.5m \\ 0.0m \\ 0.1m \end{bmatrix}$

Robot : fichier (scène vrep) **testScara.ttt**



$i, i+1$	Rot/ z_i	Trans/ x_{i+1}	Trans/ z_i	Rot/ x_{i+1}
0,1	$q1 = \theta1^*$	$a1 = L1$	$d1 = L3$	$\alpha1 = 0$
1,2	$q2 = \theta2^*$	$a2 = L2$	$d2 = 0$	$\alpha2 = 0$
2,3	$\theta3 = 0$	$a3 = 0$	$q3 = d3^*$	$\alpha3 = 0$
3,4	$q4 = \theta4^*$	$a4 = 0$	$d4 = 0$	$\alpha4 = 0$

1 MGD et MGI avec matlab/octave (calcul symbolique)

En vous référant au tableau de *Denavit-Hartenberg* du robot, adapter le programme symbolique octave

scara_MTB_symbolic_etudiant.m pour déterminer et résoudre les équations telles que l'extrémité de l'outil ait

pour coordonnées dans le repère 0: ${}^0_{outil} = \begin{bmatrix} xO_d \\ yO_d \\ zO_d \end{bmatrix}$

Pour cela compléter successivement chacune des étapes du programme, jusqu'à obtention des équations à résoudre :

étape 1 : calcul des transformations homogènes 0T_1 , 1T_2 , 2T_3 , 3T_4 , 1T_4 , 0T_4 du robot, notées respectivement T_{01} , T_{12} , T_{23} , T_{34} dans le programme

étape 2 : calcul de 1T_4 puis 0T_4 , notées T_{14} , T_{04}

étape 3 : établissement de la posture désirée 0T_4 désirée , (notée $T_{04_desiree}$)

étape 4 : essai de résolution automatique des équations : ${}^0O_4 = {}^0O_4$ désirée (ça peut marcher, car ça dépend de l'efficacité du solveur symbolique)

étape 5: essai de résolution automatique des équations : ${}^1O_4 = {}^1O_4$ désirée (qui nécessitent de déterminer l'expression de 1T_4 désirée)

étape 6: résolution semi-manuelle des équations : ${}^1O_4 = {}^1O_4$ désirée (en aidant le solveur symbolique)

étape 7: génération des fonctions matlab/octave calculant les solutions des équations

clc_MGD_scara : calcule 0T_4 en fct des coordonnées articulaires

clc_MGI_scara : calcule les coordonnées articulaires en fct de 0O_4 désirée

quelques règles à respecter :

- vérifier la cohérence des résultats obtenus avec ce que vous attendez, en particulier lorsque tous les angles sont nuls, puis lorsque $\theta_1 = 0^\circ, 90^\circ = 90^\circ$

2- chargement du modèle sous Vrep

- Si pas installé Télécharger et décompresser l'archive de **vrep**, sous votre répertoire de travail :

<http://www.coppeliarobotics.com/downloads.html>

(choisir la version : Non-limited EDUCATIONAL version. Free.)

- lancer **vrep** depuis un terminal, après vous être placés sous le répertoire d'installation:

```
cd ~/V_REP_
```

```
sh ./vrep.sh
```

- Sous vrep :

- charger la scène : **scene_Scara_MTB.ttt** (dans TD1-scara...), menu : *File->open scene*

- lancer la simulation de la scène, pour vérifier que tout fonctionne correctement : *simulation->start ..*

- éditer le script Lua simulant le robot : Tools->Scripts->Non-Threaded child script(MTB_Robot)

- Votre travail consistera à remplacer la partie du programme mitsubishi, entre les lignes

REM DEBUT DE PROGRAMME

....

GOTO DEBUT

par votre propre programme, dont le texte sera généré avec octave/matlab

(noter la présence d'une aide succincte du langage mitsubishi en début de programme).

2 Travail numérique (sous matlab/octave)

éditer (octave/matlab) le programme matlab *scara_MTB_numerique_etudiant.m*,

étape 1- adapter le programme pour générer la trajectoire et le programme du robot correspondant au cahier des charges en début de TD (tester avec peu de points au départ):

- attention aux unités
- attention aux points impossibles

étape 2 – pour tester votre programme, le copier dans le script lua (remplace les lignes REM DEBUT à GOTO DEBUT), et lancer la simulation.

Etape3- : modifier le programme pour que le carré ne change pas d'orientation lors du suivi de trajectoire